FUNDACIÓN UNIVERSITARIA COLOMBO INTERNACIONAL (UNICOLOMBO)

TECNOLOGÍA EN DESARROLLO DE SOFTWARE
SEMESTRE IV

ANÁLISIS Y DESARROLLO DE E-SERVICE: SISTEMA DE GESTIÓN DE PEDIDOS, INVENTARIO Y VENTAS PARA RESTAURANTES

Actualización tecnológica del proyecto: C# y Avalonia UI

INTEGRANTES:

Sebastian Baena – Código: 2510180035

Kenneth Suárez – Código: 2510180001

DIRECTOR:

Juan Jose Puello Beltran

Docente Asesor de Proyecto

CARTAGENA DE INDIAS D. T. Y C.
2026

RESUMEN Y PALABRAS CLAVE

El presente proyecto de aula, denominado E-Service, consiste en el análisis y desarrollo de una aplicación de escritorio orientada a apoyar la gestión operativa de restaurantes, integrando el registro de pedidos, el control de insumos, las compras y el seguimiento de transacciones. Esta versión representa una evolución respecto a la propuesta inicial: el sistema fue reescrito en C# sobre la plataforma .NET y utiliza Avalonia UI para construir una interfaz de escritorio moderna y multiplataforma. La organización del código sigue el patrón Modelo–Vista–Modelo de vista (MVVM), separando los datos, la lógica de negocio y la presentación. La información se conserva localmente mediante archivos binarios .dat y archivos de texto destinados a comprobantes de pedidos y facturas de compra; el proyecto no utiliza una base de datos relacional. Entre sus flujos principales se encuentran el acceso de empleados, la toma y confirmación de pedidos, el cobro, la administración del inventario, el registro de compras y la gestión de usuarios. El código y la documentación técnica se encuentran publicados en el repositorio del proyecto en GitHub. Este documento describe la versión actual y no la arquitectura Java/Swing planteada en la entrega anterior.

Palabras clave: E-Service, restaurantes, pedidos, inventario, ventas, C#, .NET, Avalonia UI, MVVM, persistencia local.

1. INTRODUCCIÓN

Los restaurantes realizan diariamente operaciones relacionadas con la atención de pedidos, el consumo de insumos, la compra de existencias y el registro de ventas. Cuando estas actividades se gestionan de forma aislada o manual, puede resultar más difícil mantener información consistente, identificar consumos y consultar los movimientos del negocio. Un sistema de información de escritorio puede centralizar parte de estos procesos y facilitar el trabajo del personal.

E-Service se plantea como una herramienta para integrar esas operaciones en una aplicación de escritorio. La versión inicial del proyecto contemplaba Java, Java Swing y archivos de texto plano. Durante su evolución se decidió realizar una reescritura en C# con Avalonia UI, reorganizar la solución mediante MVVM y mejorar el mecanismo de almacenamiento local. Por tanto, la presente versión debe entenderse como una actualización funcional y tecnológica del proyecto presentado previamente.

El documento presenta la problemática, la justificación, los objetivos, los referentes conceptuales, la metodología y la descripción técnica de la versión disponible en el repositorio. Las funcionalidades se describen según la documentación publicada del proyecto; los resultados cuantitativos de pruebas con usuarios no se afirman, ya que no se suministraron métricas verificadas.

2. PLANTEAMIENTO DEL PROBLEMA

La operación de un restaurante depende de la coordinación entre la toma de pedidos, la preparación de productos, el cobro y el control de los insumos utilizados. Si los registros se mantienen en medios separados, la actualización del inventario y la consulta del historial de transacciones pueden requerir tareas repetitivas y aumentar la posibilidad de inconsistencias.

En el contexto de pequeños y medianos establecimientos, una herramienta local de escritorio puede ofrecer una alternativa para registrar las operaciones esenciales sin requerir la instalación de un servidor de base de datos. E-Service reúne flujos de atención y administración en una misma aplicación, con pantallas diferenciadas según las tareas del personal de servicio y de administración.

Pregunta problema: ¿Cómo desarrollar y organizar una aplicación de escritorio multiplataforma que integre los procesos de pedidos, cobro, inventario, compras y administración de usuarios de un restaurante, utilizando C#, .NET y Avalonia UI, con persistencia local de la información?

3. JUSTIFICACIÓN

Justificación académica. El proyecto permite aplicar programación orientada a objetos, separación de responsabilidades, diseño de interfaces, enlace de datos y persistencia de información en un caso práctico. La reescritura también permite estudiar una arquitectura de aplicación más modular que la versión inicial.

Justificación operativa. La integración de pedidos, compras, inventario y transacciones busca reducir la duplicación de registros y facilitar el acceso a información útil para el personal. El sistema incluye flujos para tomar pedidos, confirmar el cobro, consultar existencias, registrar compras y administrar usuarios.

Justificación tecnológica. C# y .NET proporcionan la base de la aplicación; Avalonia UI permite desarrollar interfaces de escritorio con tecnologías XAML en distintos sistemas operativos compatibles. El almacenamiento local evita depender de un servidor de base de datos para el funcionamiento descrito en el repositorio, aunque implica que deben protegerse y respaldarse los archivos de datos del equipo.

Alcance. E-Service es una aplicación de escritorio con almacenamiento local. Este documento no afirma que incluya sincronización en la nube, operación multiusuario concurrente, facturación electrónica autorizada por la DIAN ni integración con pasarelas de pago, porque esas capacidades no están acreditadas en la documentación consultada.

4. OBJETIVOS

4.1 Objetivo general

Desarrollar y documentar una aplicación de escritorio para apoyar la gestión de pedidos, cobros, inventario, compras y usuarios en restaurantes, utilizando C#, .NET y Avalonia UI, con una arquitectura organizada mediante MVVM y almacenamiento local de datos.

4.2 Objetivos específicos

- Identificar y organizar los procesos principales del sistema: inicio de sesión, pedidos, cobro, inventario, compras y administración de empleados.
- Separar los modelos de datos, la lógica de negocio y la interfaz mediante las carpetas Models, Services, ViewModels y Views.
- Construir la interfaz con Avalonia UI y vistas XAML, utilizando enlace de datos y comandos para conectar las acciones del usuario con los modelos de vista.
- Implementar persistencia local de datos estructurados en archivos .dat y generación de comprobantes legibles en archivos .txt.
- Documentar la estructura técnica y establecer una base para realizar pruebas funcionales de los flujos principales.

5. MARCO REFERENCIAL

5.1 Sistemas de información para restaurantes

Un sistema de información recoge, procesa, almacena y presenta datos que apoyan las operaciones de una organización. En un restaurante, la relación entre pedidos, productos, recetas, inventario y transacciones permite mantener un registro operativo más coherente. La utilidad real depende de la calidad de los datos introducidos y de los procedimientos del establecimiento.

5.2 Programación orientada a objetos y C#

La programación orientada a objetos estructura el software mediante tipos y objetos que agrupan estado y comportamiento. En E-Service, los modelos representan entidades del dominio, mientras que los servicios concentran operaciones como lectura y escritura de datos, validación de acceso, gestión de insumos y registro de transacciones. C# es el lenguaje principal de la versión actual.

5.3 Avalonia UI y XAML

Avalonia UI es un marco de interfaz para aplicaciones .NET que permite crear interfaces con XAML y ejecutar aplicaciones de escritorio en más de un sistema operativo. En el proyecto, las vistas .axaml describen la presentación visual y se conectan con los modelos de vista. Esta separación ayuda a mantener el diseño de pantalla separado de la lógica de la aplicación.

5.4 Patrón Modelo–Vista–Modelo de vista (MVVM)

MVVM separa los modelos que representan datos, las vistas que muestran la interfaz y los modelos de vista que exponen datos y acciones a la pantalla. En E-Service, esta estructura facilita que los botones y controles de una vista ejecuten comandos del ViewModel y que la interfaz refleje los cambios de los datos mediante binding.

5.5 Persistencia local y estructuras de datos

La versión documentada guarda usuarios, insumos y transacciones en archivos binarios .dat mediante lectura y escritura binaria. Los comprobantes de pedidos y las facturas de compra se conservan como archivos .txt legibles. El proyecto utiliza una lista doblemente enlazada implementada en el propio código para las colecciones. La estrategia de guardado documentada utiliza archivos temporales antes de reemplazar los datos principales y conserva copias con extensión de corrupción cuando un archivo .dat no puede leerse.

5.6 Diferencias con la versión anterior

La entrega inicial describía Java, Swing y archivos .txt como mecanismo general de persistencia. La versión actual usa C#, Avalonia UI, XAML y MVVM; organiza las clases en modelos, servicios, vistas y modelos de vista; y usa archivos .dat para los datos estructurados. También conserva la idea central del proyecto: reunir las operaciones de pedidos, inventario y ventas en una aplicación local.

6. DESCRIPCIÓN FUNCIONAL Y TÉCNICA

6.1 Funcionalidades principales

- Acceso de empleados mediante una pantalla de inicio de sesión.
- Pantalla principal o lobby para orientar la navegación según el flujo de trabajo.
- Toma de pedidos por parte del personal de servicio y paso al proceso de cobro.
- Resumen de compra que agrupa unidades repetidas de un mismo producto.

- Panel de administración con acceso a inventario, compras y gestión de empleados.
- Catálogo de platos y recetas asociado al consumo de insumos.
- Registro de transacciones de ventas y compras.
- Generación de comprobantes de pedidos y facturas de compra en archivos de texto.

6.2 Organización del proyecto

La estructura publicada organiza el código en los siguientes directorios:

- Collections/: contiene DoublyLinkedList<T>, estructura de lista doblemente enlazada utilizada por el proyecto.
- Models/: contiene las clases de datos del dominio, como Cliente, Pedido, Insumo, Empleado, Plato y Transaction.
- Services/: concentra la lógica de negocio, la persistencia y servicios de apoyo, entre ellos FileDataStore, LoginService, InsumoService, PlatoService, TransactionService y CartService.
- ViewModels/: conecta los datos y servicios con las acciones disponibles en cada pantalla.
- Views/: contiene las vistas XAML; incluye pantallas de acceso, pedidos, cobro, inventario, compras y administración.
- Assets/: reúne recursos visuales como iconos e imágenes.

6.3 Flujo de uso general

El flujo documentado comienza con el inicio de sesión. A partir del acceso, el personal de servicio puede crear un pedido y continuar al cobro; el personal de administración puede consultar el panel principal y entrar a inventario, compras o administración de empleados. La navegación se realiza dentro de una ventana principal, sustituyendo la vista mostrada en lugar de abrir una ventana independiente para cada pantalla.

6.4 Almacenamiento y ubicación de los datos

En Linux, el repositorio indica que los datos se guardan en ~/.config/EService/. En Windows se utiliza %APPDATA%\\EService\\. Los archivos Data/users.dat, Data/insumos.dat y Data/transactions.dat contienen datos binarios. Las carpetas Pedidos/ y Facturas/ contienen archivos .txt legibles generados por las operaciones correspondientes. La documentación también describe una importación inicial desde archivos .txt antiguos cuando todavía no existe el archivo .dat equivalente.

6.5 Requisitos de ejecución

La documentación del repositorio indica el uso del SDK de .NET 8 para compilar y ejecutar la aplicación. El procedimiento básico publicado es abrir una terminal en la carpeta del proyecto, restaurar los paquetes y ejecutar la aplicación con dotnet restore y dotnet run. La compilación y ejecución deben comprobarse en el equipo donde se vaya a presentar el programa.

7. METODOLOGÍA

El trabajo corresponde a un desarrollo aplicado de software, con orientación descriptiva y propositiva: se aborda una necesidad operativa y se implementa una solución informática. Para documentar el producto se toma como base la descripción de la propuesta original y se contrasta con la estructura y las funcionalidades descritas en el repositorio actual.

7.1 Procedimiento

- Revisar el alcance funcional de la propuesta inicial y conservar el objetivo de integrar pedidos, inventario y ventas.

- Reestructurar la solución en C# y .NET, utilizando Avalonia UI para la presentación.
- Organizar la aplicación en Models, Services, ViewModels y Views siguiendo MVVM.
- Implementar los flujos de pedidos, cobro, administración de inventario, compras y usuarios.
- Definir persistencia local mediante archivos binarios para los datos principales y archivos de texto para comprobantes.
- Ejecutar pruebas de compilación y pruebas funcionales en el entorno de presentación; registrar los errores encontrados y corregirlos antes de la entrega.

7.2 Estrategia de validación recomendada

Para una presentación académica verificable, se recomienda probar: inicio de sesión válido e inválido; creación y confirmación de pedidos; cálculo del total; actualización del inventario conforme a las recetas; registro de una compra; alta o modificación de empleados; persistencia después de cerrar y volver a abrir el programa; y recuperación segura ante archivos de datos inválidos. Los resultados de estas pruebas deben consignarse con evidencias reales, sin sustituirlos por resultados supuestos.

8. CONCLUSIONES

E-Service evolucionó desde una propuesta basada en Java y Swing hasta una aplicación de escritorio reescrita en C# con Avalonia UI. El cambio no se limita al lenguaje: la versión actual incorpora una separación más explícita de responsabilidades mediante MVVM y organiza el código en modelos, servicios, vistas y modelos de vista.

La solución documentada integra flujos relacionados con pedidos, cobro, inventario, compras y administración de usuarios. Su persistencia local utiliza archivos binarios para datos estructurados y archivos de texto para comprobantes legibles, por lo que no depende de una base de datos relacional para el funcionamiento descrito.

El repositorio constituye la referencia técnica de la versión actual. Para sustentar una evaluación final, se recomienda acompañar este documento con capturas reales de las pantallas, una demostración de los flujos principales y los resultados de pruebas funcionales ejecutadas. No se presentan cifras de reducción de errores o mejora de productividad porque no se aportaron mediciones que permitan afirmarlas.

9. REFERENCIAS

Kenesu1963. (2026). EService: repositorio del proyecto y README. GitHub. https://github.com/Kenesu1963/EService

Kenesu1963. (2026). DOCUMENTACION_TECNICA.md.pdf [documentación técnica del proyecto]. GitHub. https://github.com/Kenesu1963/EService

Avalonia UI. (s. f.). Avalonia: documentación y características del marco de interfaz para .NET. https://github.com/AvaloniaUI/Avalonia

Microsoft. (s. f.). Documentación de .NET. https://learn.microsoft.com/dotnet/

Nota de alcance: la descripción de la arquitectura, los directorios, el almacenamiento y los flujos funcionales se elaboró a partir del README y la documentación técnica publicados en el repositorio del proyecto. Las recomendaciones de pruebas se presentan como propuestas, no como resultados ya obtenidos.

10. ANEXOS PROPUESTOS PARA LA PRESENTACIÓN

Para mantener el documento fiel al programa y facilitar su exposición, se recomienda incorporar en una siguiente revisión los siguientes anexos visuales, obtenidos directamente de la versión ejecutable:

- Anexo A. Captura de la pantalla de inicio de sesión.
- Anexo B. Captura de la pantalla principal o lobby.
- Anexo C. Captura del módulo de pedidos y del resumen de cobro.
- Anexo D. Captura del panel de inventario y registro de compras.
- Anexo E. Captura de la administración de empleados.
- Anexo F. Diagrama de arquitectura: Views → ViewModels → Services → Models / FileDataStore.
- Anexo G. Tabla de pruebas funcionales con caso, datos de entrada, resultado esperado, resultado obtenido y evidencia.